

Suplemento Milestone 3

Despliegue del modelo en Cloudflare y generación del cartel ITESO

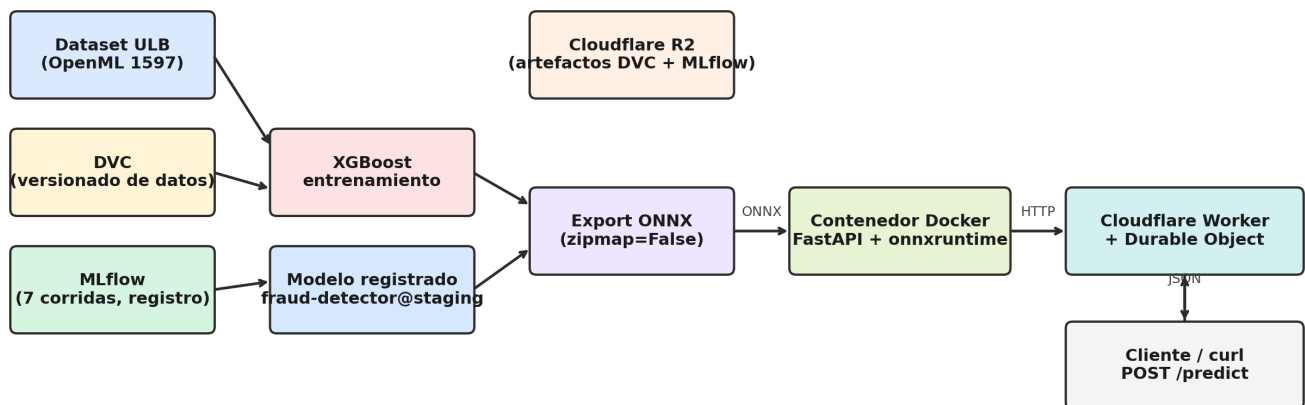
Eduardo García López · Fernando Ramos Ríos
ITESO — ISAA — Primavera 2026

Este documento complementa el reporte del Proyecto Final entregado en M2. Cubre exclusivamente las tareas del Milestone 3: (1) exportación del modelo registrado fraud-detector@staging a ONNX como artefacto portátil; (2) construcción y publicación de un contenedor de inferencia en Cloudflare; (3) verificación empírica de paridad contra el modelo local; (4) generación del cartel para el Congreso Ingenierías SUJ.

1. Arquitectura de despliegue

El modelo XGBoost ganador (xgb-d6-lr05-n500, PR-AUC 0.8819) fue exportado a ONNX y servido desde un contenedor Docker (python:3.11-slim + FastAPI + onnxruntime) publicado en el registro privado de Cloudflare. El contenedor se materializa como un Durable Object detrás de un Worker que actúa de proxy HTTP, expuesto en una URL pública *.workers.dev. La elección de Cloudflare en lugar de AWS se debe a la suspensión de la cuenta académica de uno de los autores; el pivote es parte abierta de la narrativa del proyecto.

Pipeline MLOps — entrenamiento reproducible → inferencia en el edge



Componentes

- Endpoint: <https://fraud-detector.eduardo-lalo1999.workers.dev>
- Worker (TS, ESM): `mlops/deployment/src/index.ts` — usa `@cloudflare/containers`, ruta toda petición a `getContainer(env.FRAUD_DETECTOR)`
- Container (Python): `mlops/deployment/container_src/main.py` — FastAPI con POST `/predict`, GET `/health`, GET `/`
- Modelo: ONNX 850 KB (opset 15, zipmap=False) horneado dentro de la imagen
- Instance type: basic (1/4 vCPU, 1 GiB RAM, 4 GB disco), max 3 instancias
- sleepAfter: 5 minutos (la primera petición tras inactividad paga arranque en frío de ~30 s)

2. Exportación a ONNX y paridad local

El modelo fue cargado vía `mlflow.xgboost.load_model('models:/fraud-detector@staging')`, lo que devuelve el `XGBClassifier` nativo sin necesidad de envolver/desenvolver `pyfunc`. Para que `onnxruntime` acepte los nombres de variables, renombramos los atributos del booster (V1..V28, Amount) al patrón `f0..f28` que exige `onnxmltools`. Convertimos con `skl2onnx` (registrando manualmente el converter de `XGBoost`) usando `target_opset={"": 15, "ai.onnx.ml": 3}` y `options={id(model): {"zipmap": False}}`; sin esto, el grafo ONNX habría incluido un nodo `ZipMap` incompatible con runtimes WASM/JS.

Resultado de la paridad

```
{
  "model_alias": "fraud-detector@staging",
  "model_version": 1,
  "onnx_size_kb": 850.6,
  "onnx_input_name": "input",
  "onnx_output_names": [
    "label",
    "probabilities"
  ],
  "n_test_rows": 200,
  "tolerance": 0.0001,
  "n_within_tolerance": 200,
  "max_abs_diff": 7.945345714688301e-08,
  "mean_abs_diff": 3.0091594993564286e-08,
  "best_threshold": 0.9274014830589294
}
```

Los 200 ejemplos del subconjunto de prueba coincidieron con el modelo original dentro de 1×10^{-4} ; la diferencia máxima absoluta fue $7.95e-08$, casi al ruido de punto flotante de 32 bits. ONNX queda así validado como artefacto portátil; el archivo se subió también al bucket R2 `luci-mlops-fraud/inference/` para trazabilidad.

3. Verificación contra el endpoint en producción

El script `scripts/test_inference.py` muestrea 100 filas del conjunto de prueba (incluyendo 20 fraudes reales, sobremuestreados para no quedarnos solo con negativos), las envía al endpoint, y compara la probabilidad devuelta contra `sklearn.predict_proba` del mismo modelo cargado localmente. El criterio binario es: PASS sólo si las 100 filas caen dentro de tolerancia y la decisión de clase (umbral 0.9274) coincide en todas.

Métricas de paridad

```
{
  "worker_url": "https://fraud-detector.eduardo-lalo1999.workers.dev",
  "model_alias": "fraud-detector@staging",
  "n_rows": 100,
  "n_positives_sampled": 20,
  "tolerance": 0.0001,
  "n_within_tolerance": 100,
  "max_abs_diff": 5.9329579471523175e-08,
  "mean_abs_diff": 2.6482306054731454e-08,
  "class_agreement": 100,
  "threshold": 0.9274014830589294,
  "latency_ms": {
    "p50": 252.87790850006786,
    "p95": 613.1240572501838,
    "mean": 648.7264037400157
  }
}
```

Evidencia muestra (curl_predictions.json)

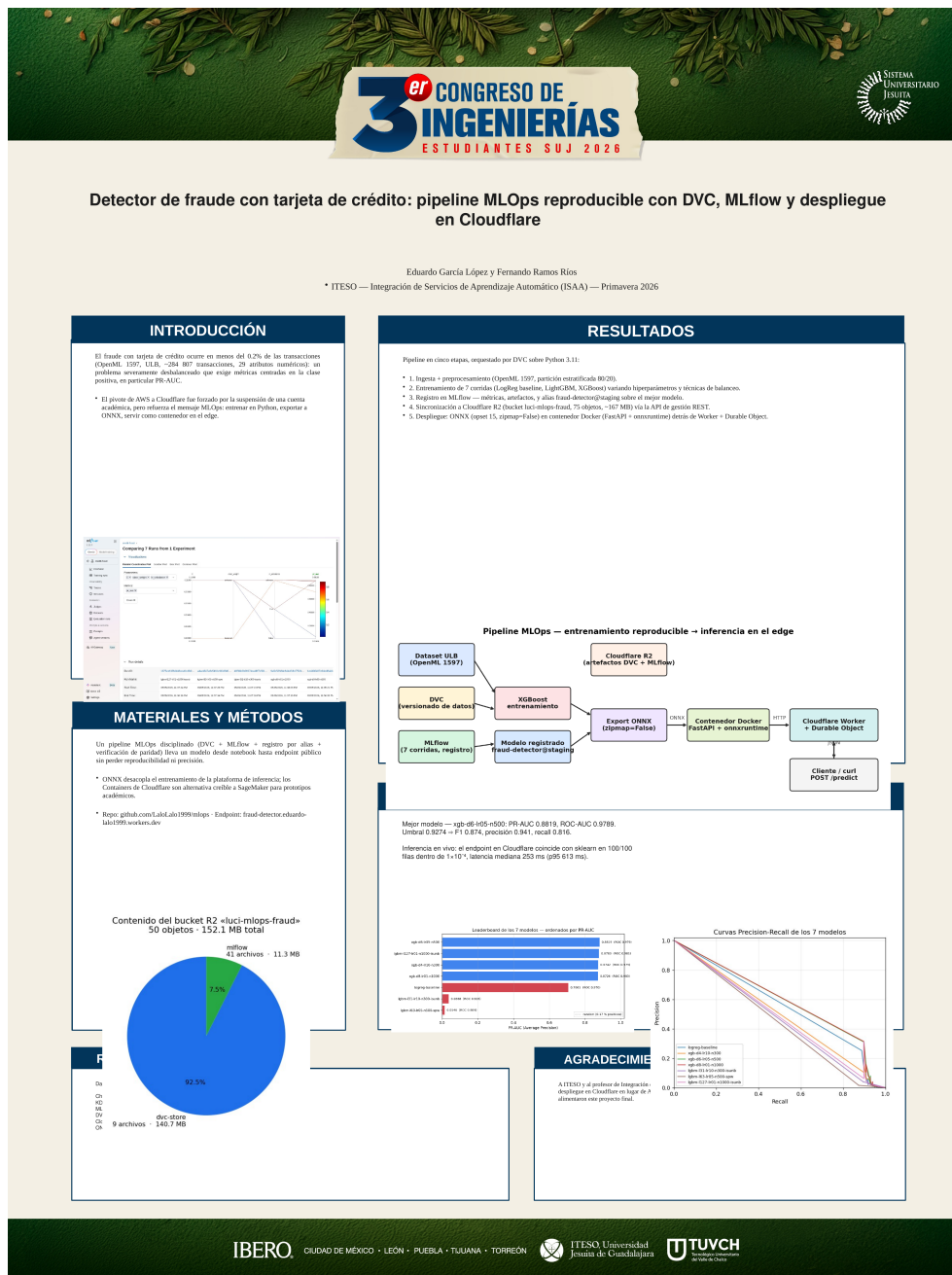
```
row 13134 (y_true=0)
  POST /predict
    → probability=2.14577e-06
    → prediction=0, latency=235.4 ms
  sklearn local: 2.1397e-06
```

```
row 19134 (y_true=0)
  POST /predict
    → probability=2.6226e-06
    → prediction=0, latency=245.6 ms
  sklearn local: 2.59704e-06
```

```
row 48974 (y_true=0)
  POST /predict
    → probability=3.21865e-06
    → prediction=0, latency=257.4 ms
  sklearn local: 3.2335e-06
```

4. Cartel para el Congreso Ingenierías SUJ

El cartel fue generado programáticamente a partir de la plantilla oficial 2026_Formato_cartel_CongresoIngSUJ.pptx (35.4 × 47.2 in, portrait, una sola diapositiva) usando python-pptx. El script scripts/generate_cartel.py respeta la tipografía y colores de la plantilla; sólo reemplaza el texto Lorem ipsum por contenido en español y agrega las cinco figuras (arquitectura, leaderboard PR-AUC, curvas PR, comparación de corridas en MLflow UI, contenido del bucket R2). La conversión a PDF se hace con libreoffice --headless --convert-to pdf; la vista previa PNG con pdftoppm a 100 dpi.



Archivos generados:

- mlops/Cartel_ProyectoFinal_EduardoGarcia_FernandoRamos.pptx
- mlops/Cartel_ProyectoFinal_EduardoGarcia_FernandoRamos.pdf
- mlops/evidence/m3/cartel_preview.png

5. Reproducir desde cero

Nota importante: el `account\_id` de Cloudflare en `deployment/wrangler.jsonc` está fijado al de los autores. Para reproducir desde otra cuenta, reemplazar ese valor y volver a desplegar. Además, la primera petición al endpoint tras una hora de inactividad puede tardar ~30 s (arranque en frío del contenedor); peticiones subsecuentes responden en menos de 500 ms.

```
# Pre-requisitos: Python 3.11 venv, Docker corriendo, wrangler 4.56+
cd mlops
source .venv/bin/activate

# 1. Exportar a ONNX y verificar paridad local
python scripts/export_onnx.py

# 2. Subir ONNX a R2 (artefacto portátil)
CLOUDFLARE_ACCOUNT_ID=<id> wrangler r2 object put \
  luci-mlops-fraud/inference/fraud-detector-v1.onnx \
  --file models/fraud-detector-v1.onnx --remote

# 3. Desplegar contenedor + Worker (build + push + deploy)
cd deployment && npm install
CLOUDFLARE_ACCOUNT_ID=<id> npx wrangler deploy

# 4. Verificar paridad contra el endpoint en vivo
WORKER_URL=https://fraud-detector.eduardo-lalo1999.workers.dev \
  python scripts/test_inference.py

# 5. Generar cartel + PDF + preview
python scripts/generate_cartel.py
```

Comprobación rápida del endpoint

```
curl -X POST https://fraud-detector.eduardo-lalo1999.workers.dev/predict \
  -H 'Content-Type: application/json' \
  -d '{"features":[-0.674,1.408,-1.111,...,23.0]}'

# → {"probability":1.55e-06,"prediction":0,
#    "threshold":0.9274014830589294,"model_version":"v1"}
```